```python
import warnings
warnings.filterwarnings("ignore")

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

# Load fraud dataset (USED FOR MODELING)
fraud_df = pd.read_csv("fraud.csv")

# Load data dictionary (ONLY FOR UNDERSTANDING)
try:
    data_dict = pd.read_excel("Data_dictionary.xlsx")
except:
    data_dict = pd.read_csv("Data_dictionary.csv")

print("Fraud dataset shape:", fraud_df.shape)
fraud_df.head()
```

```
Fraud dataset shape: (4946768, 11)
```

```
   step      type    amount     nameOrig  oldbalanceOrg
newbalanceOrig  \
0     1   PAYMENT   9839.64  C1231006815       170136.0
160296.36
1     1   PAYMENT   1864.28  C1666544295        21249.0
19384.72
2     1  TRANSFER    181.00  C1305486145          181.0
0.00
3     1  CASH_OUT    181.00   C840083671          181.0
0.00
4     1   PAYMENT  11668.14  C2048537720        41554.0
29885.86

      nameDest  oldbalanceDest  newbalanceDest  isFraud
isFlaggedFraud
0  M1979787155             0.0             0.0      0.0
```

```
0.0
1  M2044282225             0.0             0.0      0.0
0.0
2   C553264065             0.0             0.0      1.0
0.0
3    C38997010         21182.0             0.0      1.0
0.0
4  M1230701703             0.0             0.0      0.0
0.0
```

```
# Remove rows where target is missing
print("NaNs in isFraud before:", fraud_df['isFraud'].isnull().sum())

fraud_df = fraud_df.dropna(subset=['isFraud'])

print("NaNs in isFraud after:", fraud_df['isFraud'].isnull().sum())
```

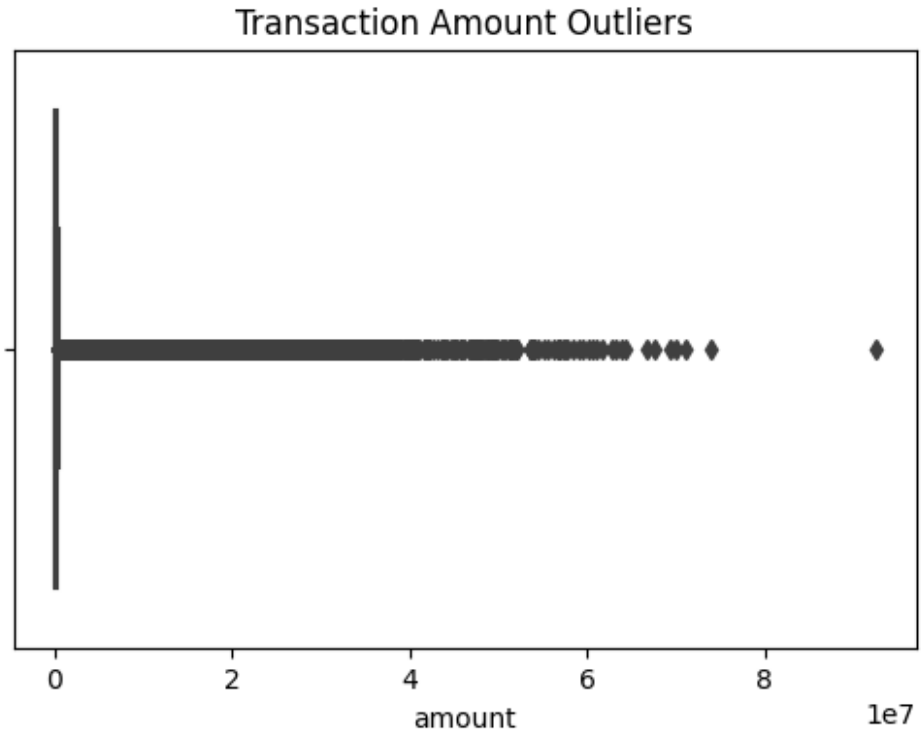
```
NaNs in isFraud before: 1
NaNs in isFraud after: 0
```

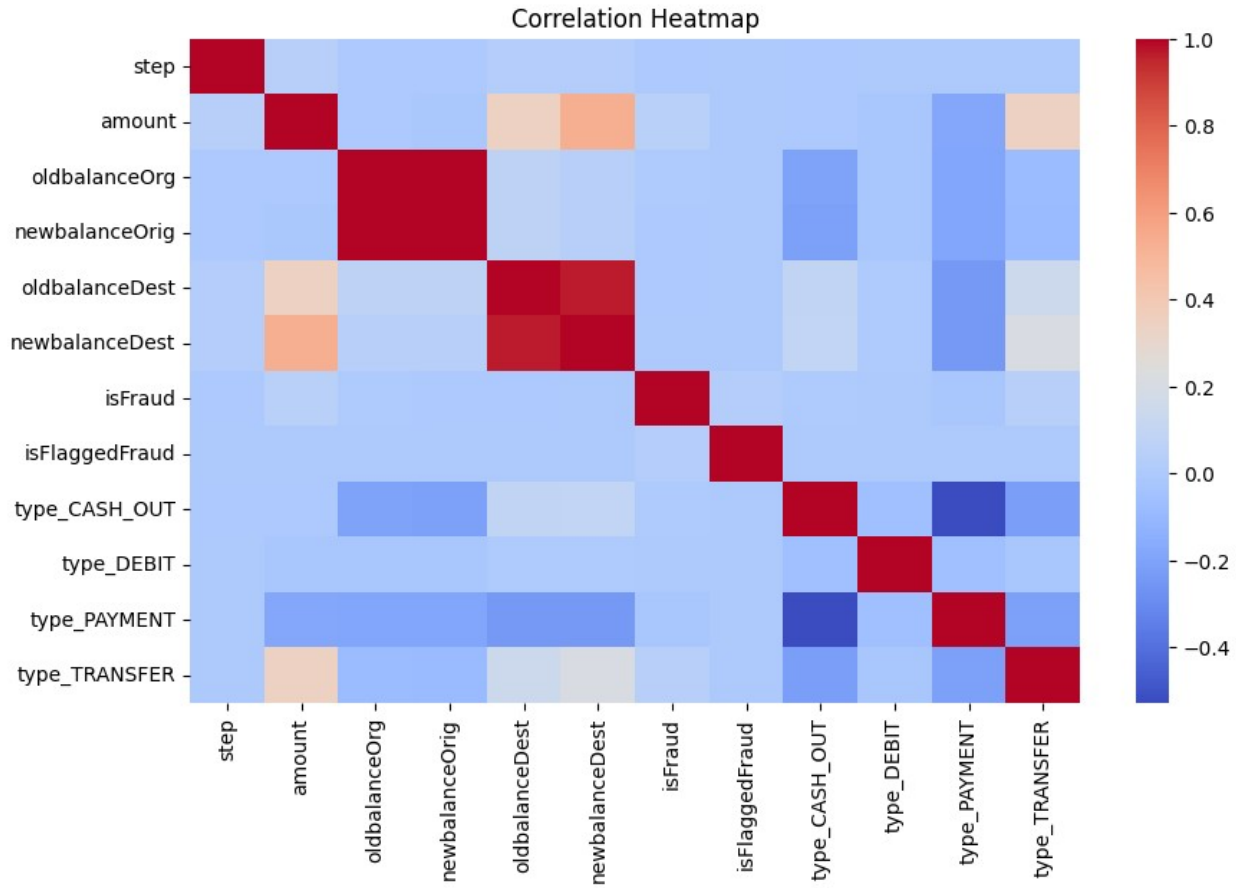
```
# Drop ID columns (not predictive)
fraud_df = fraud_df.drop(['nameOrig', 'nameDest'], axis=1)

# Encode categorical feature
fraud_df = pd.get_dummies(fraud_df, columns=['type'], drop_first=True)

fraud_df.head()
```

```
   step    amount  oldbalanceOrg  newbalanceOrig  oldbalanceDest  \
0     1   9839.64       170136.0       160296.36             0.0
1     1   1864.28        21249.0        19384.72             0.0
2     1    181.00          181.0            0.00             0.0
3     1    181.00          181.0            0.00         21182.0
4     1  11668.14        41554.0        29885.86             0.0

   newbalanceDest  isFraud  isFlaggedFraud  type_CASH_OUT  type_DEBIT
\
0             0.0      0.0             0.0          False       False

1             0.0      0.0             0.0          False       False

2             0.0      1.0             0.0          False       False

3             0.0      1.0             0.0           True       False

4             0.0      0.0             0.0          False       False


   type_PAYMENT  type_TRANSFER
0          True          False
1          True          False
```

```
2          False           True
3          False          False
4           True          False
```

```
# Outlier analysis
plt.figure(figsize=(6,4))
sns.boxplot(x=fraud_df['amount'])
plt.title("Transaction Amount Outliers")
plt.show()

# Correlation (multicollinearity)
plt.figure(figsize=(10,6))
sns.heatmap(fraud_df.corr(), cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```






```
X = fraud_df.drop('isFraud', axis=1)
y = fraud_df['isFraud']

print("Target distribution:")
print(y.value_counts())

print("NaNs in target:", y.isnull().sum())
```

```
Target distribution:
isFraud
0.0    4942864
1.0       3903
Name: count, dtype: int64
NaNs in target: 0
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

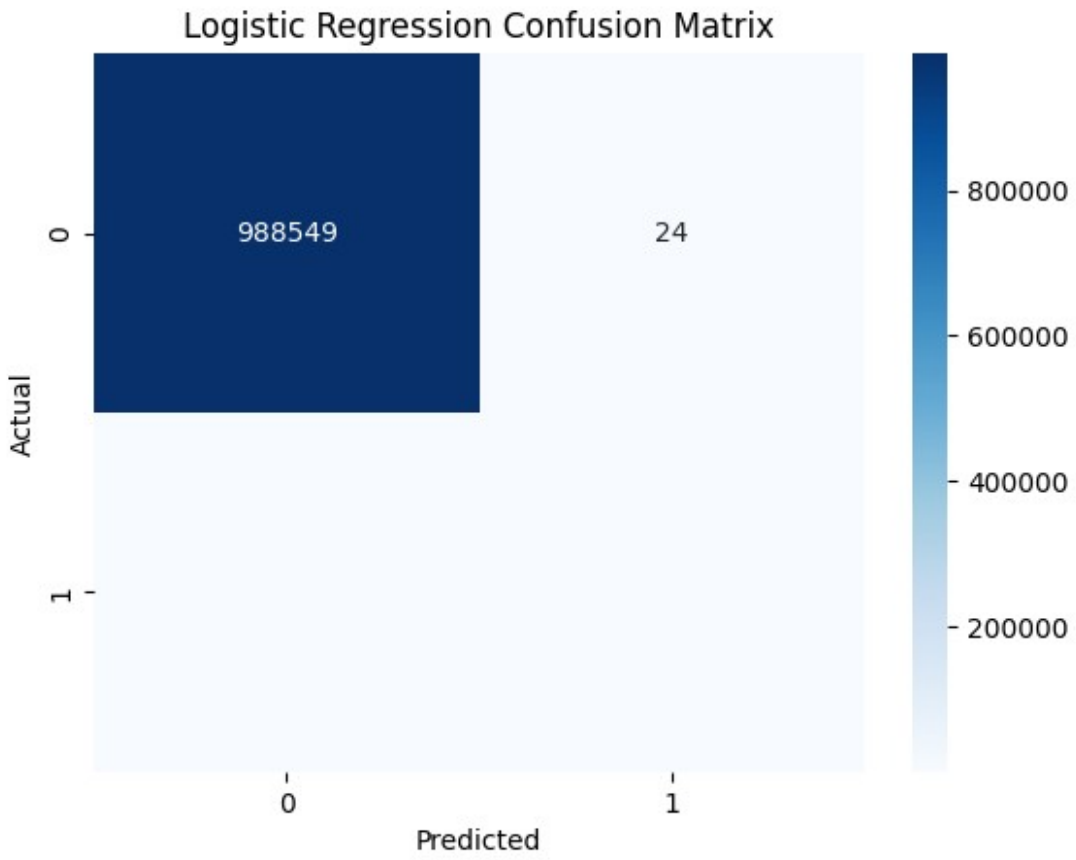
```
X_train.shape, X_test.shape
```

```
((3957413, 11), (989354, 11))
```

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

lr = LogisticRegression(max_iter=1000)
lr.fit(X_train_scaled, y_train)

y_pred_lr = lr.predict(X_test_scaled)

print("LOGISTIC REGRESSION")
print("Precision:", precision_score(y_test, y_pred_lr))
print("Recall:", recall_score(y_test, y_pred_lr))
print("F1 Score:", f1_score(y_test, y_pred_lr))
print(classification_report(y_test, y_pred_lr))
```

```
LOGISTIC REGRESSION
Precision: 0.9117647058823529
Recall: 0.3175416133162612
F1 Score: 0.47103513770180433
              precision    recall  f1-score   support

         0.0       1.00      1.00      1.00    988573
         1.0       0.91      0.32      0.47       781

    accuracy                           1.00    989354
   macro avg       0.96      0.66      0.74    989354
weighted avg       1.00      1.00      1.00    989354
```

```
sns.heatmap(
    confusion_matrix(y_test, y_pred_lr),
    annot=True,
    fmt='d',
    cmap='Blues'
)
plt.title("Logistic Regression Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```




```
rf = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    class_weight='balanced'
)

rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)

print("RANDOM FOREST")
print("Precision:", precision_score(y_test, y_pred_rf))
print("Recall:", recall_score(y_test, y_pred_rf))
print("F1 Score:", f1_score(y_test, y_pred_rf))
print(classification_report(y_test, y_pred_rf))
```

```
RANDOM FOREST
Precision: 0.9742268041237113
Recall: 0.7259923175416133
F1 Score: 0.8319882611885547
              precision    recall  f1-score   support

         0.0       1.00      1.00      1.00    988573
         1.0       0.97      0.73      0.83       781
```

```
    accuracy                           1.00    989354
   macro avg       0.99      0.86      0.92    989354
weighted avg       1.00      1.00      1.00    989354
```

```
feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": rf.feature_importances_
}).sort_values(by="Importance", ascending=False)

feature_importance.head(10)
plt.figure(figsize=(8,5))
sns.barplot(
    x="Importance",
    y="Feature",
    data=feature_importance.head(10)
)
plt.title("Top Fraud Predicting Factors")
plt.show()
```